GUÍA TÉCNICA E INDUCCIÓN

DoggyCredit

Sprint 1 — Esqueleto seguro que camina

Capacitación práctica para el primer día: arquitectura multi-tenant, contratos, seguridad, flujo de trabajo y tareas por integrante.

Versión 1.0 — 25 de septiembre de 2026

Equipo Los 5 furiosos — La Paz, Bolivia

Segunda guía de la serie, tras *MedicalSys – Guía técnica e inducción del Sprint 2*

Basada en *DoggyCredit — Visión, Requerimientos y Arquitectura v1* (mesa de diseño: Product Owner, Arquitecto, Senior Software Engineer) y en el *Product Backlog* / *Sprint 1 Backlog* en Excel. Ambos documentos siguen siendo la fuente normativa; esta guía solo la traduce en pasos concretos para el primer día.

Antes de comenzar

Esta guía asume que el equipo conoce Java, Spring y bases de datos relacionales a un nivel intermedio –lo demostraron en MedicalSys– pero que **nadie ha trabajado nunca** con esta arquitectura concreta: un monolito modular verificado con Spring Modulith, aislamiento multi-tenant con Row-Level Security de PostgreSQL, Keycloak como proveedor de identidad, un microkernel de conectores hacia fuentes financieras simuladas, y un modelo de scoring servido con ONNX dentro de la propia aplicación Java. No pretende convertirlos en expertos en un día. Su objetivo es darles un **modelo mental compartido**, una forma segura de arrancar y criterios concretos para saber si el Día 1 terminó bien.

La promesa de este sprint

Al finalizar la semana, un tenant sembrado (*Progreso Andino*) podrá iniciar sesión con un usuario real de Keycloak, dar de alta a un titular con su consentimiento, y pedir una evaluación de crédito que llama **por red** al simulador del buró (WireMock) y devuelve un **score básico** con 3 razones. En vivo se podrá demostrar que una llamada sin token responde `401`, que un usuario de otro tenant no puede ver ese titular (`404`), y que el mismo identificador de correlación aparece en la respuesta, en los logs y en la auditoría.

Cómo usar esta guía

1. Lean primero los capítulos 1 a 6 **en conjunto**. Son el vocabulario común: si cada persona entiende algo distinto por *módulo*, *tenant* o *contrato*, la integración fallará aunque el código individual sea correcto.
2. El primer día sigan el capítulo 8 como agenda operativa. No empiecen por el capítulo de su nombre sin haber pasado antes por los spikes y por la reunión de contratos.
3. Cada integrante usa luego su tutorial del capítulo 9. Las tareas hacen referencia a los identificadores del *Sprint 1 Backlog* en Excel: `SPIKE-1`, `HAB-02`, `RF-CON-01.1`, etc.
4. Mantengan abierto el Excel. Este PDF explica **cómo** trabajar; el Excel (*Product Backlog* y *Sprint 1 - Backlog*) sigue siendo la fuente de verdad de alcance, horas, estado y dependencias.
5. **Solo Nicole** (Dev4, Technical Lead y Scrum Master) inicializa los repositorios, genera los secretos y declara las variables del `.env`. El resto del equipo **clona, lee `.env.example` y nunca inventa una variable nueva** sin avisar.
6. Los contratos (OpenAPI, catálogo de errores, esquema de eventos, semilla de tenants) los publica Nicole en `contracts/`, con apoyo de esta guía. El resto del equipo los **consume y genera código a partir de ellos**; nunca escribe un DTO o una ruta a mano que ya exista en el contrato.
7. Antes de tocar una tabla o un endpoint, consulten el documento *DoggyCredit — Visión, Requerimientos y Arquitectura v1*, secciones 9 a 15. Esa es la fuente normativa de la arquitectura, las reglas de negocio y los ADR.

Reglas de seguridad de la capacitación

- Usar únicamente titulares, documentos, montos e instituciones **ficticios**. Nunca un dato de una persona real, ni siquiera “anonimizado a mano”.
- No usar credenciales reales de ningún banco, buró, billetera, proveedor de nube ni cuenta institucional. Todas las fuentes externas son simuladores WireMock en este sprint.
- Toda respuesta que use datos simulados lleva la cabecera `X-Simulated: true` y la UI muestra *“Datos simulados”*.
- Nadie sube su propio `.env` al repositorio ni comparte claves por chat. Los secretos viven solo en el `.env` local de Nicole y en los *secrets* de CI.
- `docker compose down -v` borra los datos locales. Solo se ejecuta cuando el equipo lo acuerda explícitamente.

⚠ Lo que no haremos esta semana

No construiremos Kafka, Redis, Vault/OpenBao, celdas dedicadas por tenant, autoservicio de registro, rechazo automático de crédito, el segundo banco, LightGBM ni un dashboard de equidad. Es **deuda visible y documentada** en el informe de arquitectura (§17.3), no trabajo olvidado. Si alguien "ya casi termina" algo de esa lista, se detiene y avisa: probablemente esté resolviendo el problema equivocado para este sprint.

Índice general

Antes de comenzar **2**

Cómo usar esta guía . . . 2

1 Objetivo del sprint y forma de trabajar **7**

1.1 El objetivo expresado sin tecnología . . . 7

1.2 Qué significa “Hecho” . . . 7

1.3 Capacidad y disciplina . . . 8

2 La arquitectura que nadie del equipo conoce todavía **9**

2.1 Por qué no repetimos los 12 microservicios de MedicalSys . . . 9

2.2 Qué es un “módulo” en Spring Modulith . . . 9

2.3 Qué es un *tenant* y por qué nos obsesionamos con él . . . 9

2.4 Qué es un “contrato” aquí . . . 10

2.5 Los componentes que sí vamos a tocar esta semana . . . 10

3 La arquitectura local de DoggyCredit **11**

3.1 Un solo proceso Java, muchos módulos . . . 11

3.2 Una base PostgreSQL, un esquema por módulo . . . 11

3.3 Dónde viven los contratos . . . 11

3.4 Puertos y contenedores del `docker-compose.yml` . . . 11

4 Cómo hablan los componentes entre sí **12**

4.1 Contract-first: el OpenAPI se escribe antes que el código . . . 12

4.2 Formato de error común (RFC 9457) . . . 12

4.3 Un solo id de correlación . . . 12

4.4 `Idempotency-Key` . . . 12

4.5 Simuladores externos: la regla del “sin modo local” . . . 12

5 Identidad, JWT y aislamiento por tenant **13**

5.1 Qué contiene el token . . . 13

5.2 RLS: la segunda capa de defensa, explicada paso a paso . . . 13

5.3 Rutas públicas e internas . . . 13

5.4 Qué nunca hacer . . . 13

6 La plantilla común de un módulo **14**

6.1 `api/` contra `internal/` . . . 14

6.2 Contenido mínimo de un módulo . . . 14

6.3 Convenciones de nombres . . . 14

6.4 Configuración por variables: lo que el equipo sí puede tocar . . . 14

7 Acuerdos de equipo antes de escribir negocio **16**
7.1 Reunión de contratos (día 1, primera hora) . . . 16
7.2 Cómo compartir cambios sin bloquearse . . . 16
7.3 Daily de 15 minutos . . . 16

8 Día 1: tutorial operativo paso a paso **17**
8.1 Resultado obligatorio al cerrar el día . . . 17
8.2 Agenda del día . . . 17
8.3 Paso 1 — Verificar herramientas . . . 17
8.4 Paso 2 — Recorrer el Excel . . . 17
8.5 Paso 3 — Clonar el repositorio . . . 17
8.6 Paso 4 — Levantar infraestructura primero . . . 18
8.7 Paso 5 — Trabajar el *spike* o la tarea asignada . . . 18
8.8 Paso 6 — Verificar aislamiento (todos, no solo quien lo construyó) . . . 18
8.9 Paso 7 — Cierre del día . . . 18

9 Tutorial del Día 1 por integrante **19**
9.1 Juanjo — Plataforma, seguridad y tenancy . . . 19
9.2 Alvaro — Integraciones, consentimiento y datos sintéticos . . . 20
9.3 Jhon — Dominio de crédito: solicitante, score y auditoría . . . 21
9.4 Nicole — ML, Technical Lead y Scrum Master . . . 22
9.5 Franco — Identidad, frontend y plataforma de despliegue . . . 23

10 Operación diaria con Docker Compose **25**
10.1 Rutina de inicio . . . 25
10.2 Rutina después de cambiar código . . . 25
10.3 Rutina de diagnóstico . . . 25
10.4 Volúmenes y reinicio limpio . . . 25
10.5 Logs y correlación útiles . . . 25

11 Plan resumido de los Días 2 a 5 **26**

12 Troubleshooting para principiantes **27**
12.1 Árbol de decisión rápido . . . 27
12.2 Cuándo pedir ayuda . . . 27

13 Definition of Done y preparación de la Review **28**
13.1 Checklist por tarea . . . 28
13.2 Checklist del incremento (cierre del Sprint 1) . . . 28
13.3 Guion de la Review del Sprint 1 . . . 28

A Hoja rápida de comandos **29**

B Variables y nombres recomendados **29**

C Glosario mínimo **29**

Referencias y fuentes **30**

1. Objetivo del sprint y forma de trabajar

Scrum no exige crear un producto distinto cada semana. Exige que el sprint produzca un incremento *usable, integrado y revisable*. Para DoggyCredit aplicamos la misma regla que en MedicalSys, un poco más estricta: cada Sprint Review debe mostrar un flujo que atraviese la seguridad, la base de datos y al menos una fuente simulada, y que tenga sentido para alguien fuera del equipo.

1.1 El objetivo expresado sin tecnología

El objetivo del Sprint 1 no es levantar Keycloak, PostgreSQL y seis simuladores. El objetivo es demostrar que **una evaluación de crédito puede pedirse, aislarse por institución y responder con una razón entendible**, todo verificado en vivo. La infraestructura es el medio. Si el viernes hay diez tests verdes pero nadie puede iniciar sesión y pedir una evaluación real, el sprint no está terminado.

Etapa	Actor	Resultado visible	Módulos principales
1. Preparar el tenant	Nicole (TL)	Tenant *Progreso Andino* sembrado, con usuarios Keycloak	`tenancy`, Keycloak
2. Iniciar sesión	Analista	Token con `tenant_id` correcto	Keycloak, `shared`
3. Dar consentimiento	Analista	Consentimiento vigente para el buró	`consent`
4. Pedir evaluación	Analista	Evaluación `201` con score y 3 razones	`assessment`, `integration`
5. Consultar el buró	Sistema	Llamada real por red al simulador	`integration`, WireMock
6. Puntuar	Sistema	Score básico (scorecard)	`scoring`
7. Verificar aislamiento	QA	404 entre tenants, 401 sin token	`shared`, RLS
8. Auditar	Oficial de cumplimiento	Registro de acceso, sin poder editarlo	`audit`

1.2 Qué significa “Hecho”

Una tarea no está **Hecha** porque el controlador responde en la computadora de quien la escribió. Debe cumplir la *Definition of Done* del Sprint 1 Backlog y, como mínimo: migración Flyway repetible desde cero, prueba de integración con Testcontainers (no con dobles en memoria), contrato actualizado si tocó la API, meta-test de aislamiento si tocó una tabla con `tenant_id`, sin datos reales en ningún lado, y una llamada real desde quien la consume.

> *“En mi módulo funciona” no es una frase aceptada esta semana. La respuesta correcta es: “funciona desde el Compose completo, con una base limpia y con evidencia repetible”.*

1.3 Capacidad y disciplina

El Sprint 1 Backlog compromete **55 horas efectivas**, repartidas de forma desigual porque este sprint es pesado en plataforma (ver el reparto exacto en el capítulo 9). El resto de la semana se va en reuniones, aprendizaje de herramientas nuevas, revisiones de código y esperas. No llenen ese margen con funciones nuevas: el margen es lo que hace posible terminar.

- **Límite de trabajo en curso:** una tarea principal por persona a la vez.
- Si alguien queda bloqueado más de 30 minutos, actualiza el Excel y pide ayuda en el chat del equipo.
- **Integración dos veces al día:** antes del almuerzo y antes del cierre, sobre `main` con Compose completo levantado.
- Los 6 *spikes* (capítulo 9) tienen **timebox estricto**: si se agota el tiempo sin cumplir el criterio de salida, se activa el plan B documentado en el Sprint 1 Backlog. No se "sigue insistiendo" una hora más.
- El viernes se demuestra desde datos limpios (`docker compose down -v && docker compose up`). No se prepara la base a mano minutos antes de la Review.

2. La arquitectura que nadie del equipo conoce todavía

Esta es la sección más importante del documento. Léanla despacio, aunque ya sepan Spring.

2.1 Por qué no repetimos los 12 microservicios de MedicalSys

En MedicalSys aprendimos —a veces por las malas— que cada frontera de red entre servicios cuesta un contrato que puede divergir, una superficie de autorización nueva y una fuente de identidad que se puede desincronizar. Las seis lecciones de la revisión post-demo de MedicalSys nacieron casi todas en esas fronteras. Para DoggyCredit, la mesa de diseño (Product Owner, Arquitecto y Senior Engineer) decidió lo contrario: **un solo proceso Java**, con módulos cuyos límites se verifican automáticamente, en vez de doce procesos independientes.

Modelo mental

Piensen en DoggyCredit como **un edificio con muchas oficinas** (los módulos), no como **muchas casas separadas** (los microservicios de MedicalSys). Las oficinas comparten el mismo edificio, la misma electricidad y la misma puerta de entrada (el proceso Java, la base de datos), pero cada una tiene su propia llave: no se puede entrar a la oficina de al lado sin pasar por la puerta principal.

2.2 Qué es un “módulo” en Spring Modulith

Spring Modulith es una librería que verifica, en cada build, que los módulos respeten sus fronteras. Cada módulo (`tenancy`, `consent`, `integration`, `scoring`, `assessment`, etc.) es un paquete Java con esta forma:

```
com.doggycredit.consent/
  ConsentApi.java        // lo unico visible desde otros modulos
  ConsentGranted.java    // evento publico (record)
  internal/
    ConsentService.java  // logica de negocio, INVISIBLE fuera del modulo
    ConsentRepository.java
    ConsentEntity.java
```

Todo lo que está bajo `internal/` es invisible para el resto del código: si alguien de otro módulo intenta importar `ConsentRepository`, el build **falla**. La única forma de hablar con `consent` desde afuera es a través de `ConsentApi` o de los eventos que publica. Esto se verifica con una prueba automática (`ApplicationModules.verify()`) que corre en cada *pull request*: nadie necesita “acordarse” de respetar la frontera, el CI la hace cumplir.

No negociable

Está prohibido hacer un `JOIN` en SQL entre tablas de dos módulos distintos, y está prohibido importar una clase de `internal/` de otro módulo. Si dos módulos necesitan el mismo dato, uno le pregunta al otro a través de su `Api`, o reacciona a un evento que el otro publicó.

2.3 Qué es un *tenant* y por qué nos obsesionamos con él

Un **tenant** es una institución cliente de DoggyCredit (un banco, una cooperativa, una fintech). **Todos** los tenants comparten las mismas tablas de PostgreSQL —no hay una base de datos por institución, como en MedicalSys—, porque un tenant gratuito debe costarnos casi cero. Eso significa que, si algo falla, los datos de una institución podrían quedar visibles para otra. Es exactamente lo que le pasó a MedicalSys (fugas de autorización en tres servicios), y es la razón por la que esta arquitectura defiende el aislamiento con **dos capas independientes** en vez de una sola:

1. **La aplicación filtra por `tenant_id`.** Cada consulta que Spring genera agrega automáticamente “`WHERE`

`tenant_id = ...`” gracias a Hibernate `@TenantId`.

2. **PostgreSQL filtra otra vez, con Row-Level Security (RLS).** Aunque la aplicación se equivoque, la base de datos **no deja ver ni escribir** una fila que no pertenezca al tenant activo en esa conexión.

Modelo mental

Piensen en RLS como un **portero en cada mesa del restaurante**, no solo en la puerta. Aunque alguien logre entrar al restaurante (pase la autenticación), cada mesa (cada fila de cada tabla) tiene su propio portero que solo deja sentarse a quien tiene la pulsera correcta (el `tenant_id` de la sesión). Si no traés pulsera, **no ves ninguna mesa**, ni siquiera para contar cuántas hay.

La regla de oro del tenant

El `tenant_id` **nunca** se lee de una cabecera HTTP, de un parámetro de la URL ni del cuerpo de la petición. Sale **siempre y únicamente** del token JWT que Keycloak firmó. Si en algún momento ven código que hace `request.getHeader("X-Tenant-Id")`, deténganse: eso fue exactamente el fallo de MedicalSys y aquí está prohibido por una regla de arquitectura verificada en el CI.

2.4 Qué es un “contrato” aquí

Un **contrato** es cualquier documento que dos partes del sistema deben respetar sin negociar en el momento: la especificación OpenAPI de la API, el catálogo único de códigos de error, el esquema de un evento, o el esquema del *Perfil Financiero 360*. Los contratos se escriben **antes** que el código que los implementa (*contract-first*), viven en la carpeta `contracts/` del repositorio, y Nicole es quien los publica y los versiona. El resto del equipo genera código automáticamente a partir de ellos: nadie escribe a mano un DTO que el contrato ya describe.

2.5 Los componentes que sí vamos a tocar esta semana

Componente	Qué es, para quién no lo conoce
`doggycredit-api`	El único proceso Java (Spring Boot 4.1 + Spring Modulith). Todos los módulos del capítulo 10 del informe viven aquí.
Keycloak	Servidor de identidad. Emite los tokens JWT y sabe a qué `tenant` pertenece cada usuario. Nadie del equipo construye login propio.
PostgreSQL 17	Una sola base, un esquema por módulo, con RLS activada en toda tabla que tenga `tenant_id`.
WireMock (buró)	Un simulador HTTP externo que imita al buró de crédito. Corre como contenedor aparte: **nunca** como un objeto falso dentro del proceso Java.
Caddy	Solo hace HTTPS y sirve la SPA. **No autoriza nada**: la seguridad vive en la aplicación, nunca en un gateway propio.
LGTM (Grafana)	Trazas, métricas y logs, todos con el mismo id de correlación.

3. La arquitectura local de DoggyCredit

3.1 Un solo proceso Java, muchos módulos

`doggycredit-api` es un proyecto Maven único. Dentro conviven 11 módulos: `shared`, `tenancy`, `billing`, `consent`, `customer` (Perfil 360), `integration`, `scoring`, `recommendation`, `assessment`, `analytics` y `audit`. En el Sprint 1 solo tocamos una parte de cada uno (ver capítulo 9); el resto se completa en los sprints 2 y 3.

3.2 Una base PostgreSQL, un esquema por módulo

No hay "la base de `consent`" y "la base de `scoring`" como bases físicas separadas (eso era MedicalSys). Hay **una** base PostgreSQL con un *esquema* por módulo (`consent.*`, `scoring.*`, `audit.*`...). Esto permite transacciones que cruzan módulos cuando hace falta (por ejemplo, registrar la evaluación y su auditoría en la misma transacción), sin perder la separación de responsabilidades.

3.3 Dónde viven los contratos

```
doggycredit/
  contracts/
    openapi/doggycredit-api.v1.yaml   // la API publica, fuente de verdad
    connectors/bureau.v1.yaml         // contrato del simulador de buro
    errors/catalog.yaml               // catalogo unico de codigos de error
    events/*.schema.json              // eventos publicados (outbox)
  backend/         // doggycredit-api (Maven)
  web/              // SPA Angular
  simulators/wiremock/
  seed/tenants.yaml // UNICA fuente de tenants e IDs de demo
  infra/compose.yaml
```

Atención

Si su tarea necesita un campo que el contrato todavía no tiene, **no lo agreguen a mano** en el DTO. Avisen en el daily o en el chat: el contrato se actualiza primero, en un PR aparte, y de ahí se regenera el código en ambos lados.

3.4 Puertos y contenedores del `docker-compose.yml`

Contenedor	Puerto host	Para qué
`caddy`	443	Punto de entrada único: HTTPS, sirve la SPA y enruta a la API
`doggycredit-api`	(interno)	Nunca se publica directo al host; solo a través de Caddy
`keycloak`	8081	Consola de administración y emisión de tokens
`postgres`	5432	Una base, RLS activada
`wiremock-bureau`	8089	Simulador del buró (Sprint 1). Banco y billetera se suman en el Sprint 2
`otel-lgtm`	3000	Grafana: trazas, métricas y logs correlacionados

Nota

Solo Caddy publica un puerto público. Un script del CI revisa `docker compose config` y falla si alguien publica el puerto interno de la API o de Postgres "para probar más rápido".

4. Cómo hablan los componentes entre sí

4.1 Contract-first: el OpenAPI se escribe antes que el código

La lección 1 de MedicalSys fue que la demo no probó integración real: los contratos HTTP divergieron entre quien los escribió y quien los consumió. Aquí lo evitamos con una regla simple: **nadie escribe un controlador Spring o un servicio Angular a mano** para un endpoint nuevo. Primero se actualiza `contracts/openapi/doggycredit-api.v1.yaml`, y de ahí `openapi-generator` produce la interfaz del controlador (Java) y el cliente (TypeScript). El PR de contrato se fusiona antes de que nadie más dependa de él.

4.2 Formato de error común (RFC 9457)

Toda respuesta de error, en cualquier endpoint, tiene esta forma exacta:

```
{
  "type": "https://docs.doggycredit.example/errors/consent-required",
  "title": "Consentimiento requerido",
  "status": 422,
  "detail": "No hay consentimiento vigente para la fuente BURO.",
  "instance": "/api/v1/credit-assessments",
  "code": "CONSENT_REQUIRED",
  "correlationId": "0199f0c1-7a2b-7c3d-9e4f-5a6b7c8d9e0f",
  "timestamp": "2026-09-29T14:03:11Z"
}
```

El campo `code` sale siempre de `contracts/errors/catalog.yaml`, nunca de un texto libre. Si su tarea necesita un error nuevo, se agrega ahí primero.

4.3 Un solo id de correlación

La lección 6 de MedicalSys fue tener **dos** formatos de error y ninguna forma confiable de seguir una petición entre logs. Aquí hay un único id: el *trace-id* de OpenTelemetry (estándar W3C), expuesto en la cabecera `X-Correlation-Id`. Ese mismo valor aparece en la respuesta, en cada línea de log y en la fila de auditoría de esa operación. Si alguien ve dos ids distintos para la misma petición, es un bug de severidad alta.

4.4 `Idempotency-Key`

Todo `POST` que consume cuota (crear una evaluación, por ejemplo) exige una cabecera `Idempotency-Key`. Repetir la misma petición con la misma clave devuelve la **misma** respuesta y **no vuelve a descontar cuota**. Esto se prueba explícitamente en el Sprint 3, pero el diseño del endpoint debe contemplarlo desde el Sprint 1.

4.5 Simuladores externos: la regla del “sin modo local”

El buró, el banco y la billetera son, en este MVP, contenedores WireMock **externos** con su propio contrato (el buró responde XML, a propósito, para que la capa de traducción sea real). **Está prohibido** programar un “modo local” o un doble en memoria que reemplace esa llamada dentro del proceso de `doggycredit-api`. La única excepción son las pruebas unitarias (no las de integración ni la demo).

No negociable

Si el simulador del buró está apagado, la evaluación debe salir marcada `PARCIAL` —nunca inventar una respuesta, nunca preaprobar. Esto es lo que en el informe se llama RN-15 y tiene una prueba dedicada.

5. Identidad, JWT y aislamiento por tenant

5.1 Qué contiene el token

Keycloak emite un JWT que incluye, entre otros, el claim `tenant_id` (el UUID de la institución) y los roles del usuario. La SPA lo obtiene con el flujo OIDC + PKCE; los sistemas de un tenant (integradores) usan *client credentials* con un cliente OAuth2 propio por tenant.

```
{
  "sub": "usr_8f2a...",
  "tenant_id": "9c7e7c1a-...-b2d4",
  "roles": ["ANALYST"],
  "exp": 1780000000
}
```

5.2 RLS: la segunda capa de defensa, explicada paso a paso

1. Al validar el JWT, un filtro de Spring Security crea un `TenantContext` inmutable con el `tenant_id` del token.
2. Al abrir cada transacción, se ejecuta `SELECT set_config('app.tenant_id', :tenant, true)` —el `true` final hace que el valor dure solo esa transacción, lo cual es seguro incluso con un *pool* de conexiones compartido (PgBouncer en modo transacción).
3. Cada tabla con `tenant_id` tiene una política RLS como esta, aplicada con `FORCE ROW LEVEL SECURITY` (por lo que ni siquiera el dueño de la tabla se salta el filtro):

```
ALTER TABLE consent.consent_grant ENABLE ROW LEVEL SECURITY;
ALTER TABLE consent.consent_grant FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON consent.consent_grant
  USING      (tenant_id = current_setting('app.tenant_id', true)::uuid)
  WITH CHECK (tenant_id = current_setting('app.tenant_id', true)::uuid);
```

Fallo cerrado

Si nadie fija `app.tenant_id` (por ejemplo, un hilo asíncrono que olvidó propagar el contexto), la comparación da `NULL` y PostgreSQL **no devuelve ninguna fila**. No hay forma de “ver todo por accidente”.

5.3 Rutas públicas e internas

Por defecto, **toda** ruta exige un token válido (*deny-by-default*). Solo hay una lista blanca explícita para salud (`/actuator/health`) y documentación (`/docs`). Un meta-test recorre el catálogo completo de rutas registradas en cada *pull request* y falla si alguna responde algo distinto de `401` sin token.

5.4 Qué nunca hacer

No negociable

- Nunca leer el tenant de una cabecera, de un parámetro de ruta o del cuerpo de una petición pública.
- Nunca usar el rol de base de datos dueño de las tablas para que la aplicación se conecte (ese rol puede saltarse RLS). La aplicación usa un rol sin `BYPASSRLS` y sin ser `OWNER`.
- Nunca comentar o desactivar una política RLS “para probar más rápido”. Una tabla nueva sin RLS rompe el build (es una *fitness function*, no una sugerencia).

6. La plantilla común de un módulo

6.1 `api/` contra `internal/`

Todo módulo nuevo sigue exactamente la misma forma que vimos en el capítulo 2. Antes de escribir la primera línea de un módulo, revisen si ya existe su carpeta (Nicole crea el esqueleto de los 11 módulos en `HAB-01`); si no existe la parte que necesitan, se agrega siguiendo el mismo patrón, nunca uno distinto "porque es más rápido".

6.2 Contenido mínimo de un módulo

- Una entidad JPA con `tenant_id` y las columnas obligatorias: `id` (UUIDv7), `tenant_id`, `created_at`, `updated_at`, `version` (bloqueo optimista).
- Un repositorio Spring Data con `@TenantId` aplicado a la entidad.
- Una migración Flyway con nombre por marca de tiempo: `V2026_09_29_1030__consent_create_grant.sql` (el formato con fecha evita choques de numeración entre 5 personas trabajando el mismo día).
- La política RLS de la nueva tabla, en la misma migración.
- Al menos una prueba de integración con Testcontainers.

6.3 Convenciones de nombres

Elemento	Convención
Código e identificadores	Inglés (`CreditAssessment`, no `EvaluacionCredito`)
Documentación, ADR y UI	Español
Esquemas y tablas	`snake_case` singular; el esquema = el nombre del módulo (`consent.consent_grant`)
Eventos	Sustantivo + verbo en pasado, versionado (`CreditAssessmentCompleted` v1)
Códigos de error	`AREA_MOTIVO` (`CONSENT_REQUIRED`, `QUOTA_EXCEEDED`)
Variables de entorno	`DC_<MODULO>_<CLAVE>` (`DC_CONSENT_DEFAULT_VALIDITY_DAYS`)
Roles	Iguales en Keycloak, backend y UI: `TENANT_ADMIN`, `ANALYST`, `API_CLIENT`, `PLATFORM_ADMIN`

⚠ Atención

En MedicalSys los roles se llamaban distinto en la guía y en el código. Aquí la lista de roles vive **una sola vez**, en `seed/tenants.yaml`, y todo lo demás la referencia.

6.4 Configuración por variables: lo que el equipo sí puede tocar

Cada módulo lee su configuración de variables de entorno, nunca de valores metidos a mano en el código. El repositorio trae un `.env.example` **sin secretos**, con la forma de cada variable documentada. El equipo puede copiarlo a `.env` local para levantar Compose, pero:

⛔ Solo Nicole

Las claves reales (`JWT_SECRET` de Keycloak si aplica, la clave maestra de cifrado, cualquier credencial) las genera y las declara **únicamente Nicole**, en su `.env` local y en los *secrets* de CI. El resto del equipo usa los valores de ejemplo del `.env.example` para desarrollo local, y **nunca** sube un `.env` con valores

reales al repositorio (`.gitignore` ya lo bloquea; `gitleaks` lo verifica en cada PR).

7. Acuerdos de equipo antes de escribir negocio

7.1 Reunión de contratos (día 1, primera hora)

Antes de que cualquiera escriba una historia de negocio, el equipo se reúne 30–45 minutos para acordar: el esquema del *Perfil Financiero 360 v1* (lo trae esbozado Nicole, el equipo lo revisa), los nombres exactos de los eventos del Sprint 1, y quién toca qué carpeta primero para evitar bloqueos. Nicole modera esta reunión como Scrum Master y publica el resultado en `contracts/` antes del mediodía.

7.2 Cómo compartir cambios sin bloquearse

- **Contrato primero, en paralelo después.** El PR de un cambio de contrato (OpenAPI o esquema de evento) se fusiona el mismo día. Quien lo consume trabaja contra el cliente generado y un mock **generado desde ese contrato**, nunca contra uno escrito a mano.
- **Trunk-based:** ramas cortas (`feat/DC-101-consent-api`), de un día como máximo, *squash merge* hacia `main`. Nada de una rama `develop` que se desincroniza —fue justamente parte del problema en MedicalSys.
- **PR de hasta 400 líneas** (sin contar código generado), con CI verde y una aprobación. Dos aprobaciones para cambios en `shared`, `tenancy`, seguridad o `contracts/`.
- Lo incompleto se integra detrás de una bandera de configuración, no en una rama que vive días sin fusionarse.

7.3 Daily de 15 minutos

Todos los días, mismo horario. Cada persona dice: qué terminó, qué va a hacer, qué la bloquea y si tiene un PR de contrato esperando revisión. Los bloqueos de más de 30 minutos se resuelven fuera del daily, no durante.

8. Día 1: tutorial operativo paso a paso

8.1 Resultado obligatorio al cerrar el día

Listo

Al final del Día 1, cada persona debe tener el repositorio clonado, el `.env` local funcionando (a partir de `.env.example`), `docker compose up` levantando sin errores, y su *spike* del capítulo 9 avanzando según el timebox. No hace falta que el walking skeleton completo funcione el día 1 —eso cierra el Sprint 1 entero— pero sí que **nadie** se quede bloqueado por el entorno.

8.2 Agenda del día

Hora aprox.	Actividad
09:00	Reunión de contratos (§7.1). Nicole ya inicializó los repositorios antes de esta reunión.
09:45	Planning del Sprint 1: repaso del Sprint 1 Backlog, cada quien confirma su tarea del capítulo 9.
10:15	Paso 1 a 4 de este capítulo: herramientas, Excel, clonar, levantar infraestructura.
11:00 – 17:00	Trabajo individual en el *spike* y las tareas asignadas (capítulo 9), con pairing donde el Excel lo indica.
17:00	Daily de cierre: qué spike salió, qué plan B se activó si alguno falló, qué queda para mañana.

8.3 Paso 1 — Verificar herramientas

```
java -version      // 21.x
mvn -version        // Maven, con el JDK 21 activo
node -v && npm -v   // Node 22+ para Angular 22
docker --version && docker compose version
git --version
```

Si algo falta, se instala **antes** de tocar el repositorio. No es tiempo perdido: es tiempo que se ahorra el resto de la semana.

8.4 Paso 2 — Recorrer el Excel

Abran `DoggyCredit_Backlogs.xlsx` y ubiquen su fila en la hoja *Sprint 1 - Backlog*. Lean la columna *Definition of Done* de cada tarea antes de empezar a programar: ahí está el criterio real de “terminado”, no en lo que cada quien imagine.

8.5 Paso 3 — Clonar el repositorio

```
git clone <url-del-repo> doggycredit
cd doggycredit
cp .env.example .env   // valores de ejemplo, sin secretos
```

> **Nota**
>
> El repositorio, el `seed/tenants.yaml` inicial y la estructura de `contracts/` ya los dejó preparados Nicole antes de esta reunión (tarea `HAB-01`). El resto del equipo no inicializa un proyecto nuevo: clona y sigue.

8.6 Paso 4 — Levantar infraestructura primero

```
docker compose up -d postgres keycloak wiremock-bureau otel-lgtm
docker compose ps   // los 4 deben quedar "healthy", no solo "running"
```

Un contenedor “running” puede seguir arrancando por dentro. Un contenedor “healthy” ya pasó su `healthcheck`. No arranquen `doggycredit-api` hasta que Postgres y Keycloak estén `healthy`.

8.7 Paso 5 — Trabajar el *spike* o la tarea asignada

Sigan la sección que les corresponde en el capítulo 9. Si su tarea depende de otra que todavía no está lista (columna *Dependencias* del Excel), trabajen contra el contrato y un mock generado, nunca esperando sentados.

8.8 Paso 6 — Verificar aislamiento (todos, no solo quien lo construyó)

Antes de cerrar el día, cualquiera que haya tocado una tabla nueva corre esta prueba manual, aunque sea rudimentaria:

```
# con un token del tenant A
curl -H "Authorization: Bearer $TOKEN_A" https://localhost/api/v1/algun-recurso
# con un token del tenant B, pidiendo el mismo recurso -> debe dar 404
curl -H "Authorization: Bearer $TOKEN_B" https://localhost/api/v1/algun-recurso/<id-de-A>
```

8.9 Paso 7 — Cierre del día

Actualicen la columna *Estado* del Excel (*Por hacer* / *En progreso* / *Bloqueado* / *Hecho*), dejen un comentario de una línea si algo quedó a medias, y lleven al daily de cierre cualquier plan B que se haya activado.

9. Tutorial del Día 1 por integrante

Las tareas de este capítulo son exactamente las del *Sprint 1 - Backlog* en Excel, con un único ajuste respecto a esa hoja: `HAB-01` (inicializar el monorepo, los contratos base y la semilla) y `HAB-07` (esbozo del Perfil Financiero 360) pasan de Juanjo a **Nicole**, porque ella asumió explícitamente la inicialización de los repositorios y la publicación de los contratos como Technical Lead. El total del sprint sigue siendo 55 horas repartidas entre el equipo; lo único que cambia es quién hace esas dos tareas puntuales.

Integrante	Foco (dueño de)	Horas	Nota
Juanjo (Juan José Claros)	Plataforma: `shared`, `tenancy`, seguridad	12 h	§9.1
Alvaro (Alvaro Torrez)	Integraciones: `integration`, `consent`	11 h	§9.2
Jhon (Jhon Huayhua)	Dominio de crédito: `assessment`, `scoring`, `audit`	6 h	§9.3
Nicole (Nicole Rodas)	ML (`scoring`/ONNX) + Technical Lead y Scrum Master	12 h + TL	§9.4
Franco (Franco Parra)	Frontend, identidad, plataforma de despliegue	14 h	§9.5

Cómo leer cada sección

Cada tarea trae su identificador del Excel entre paréntesis. La *Definition of Done* de cada una es la misma que aparece en la hoja *Sprint 1 - Backlog*; aquí se agrega el “cómo empezar” que el Excel no tiene espacio para mostrar.

9.1 Juanjo — Plataforma, seguridad y tenancy

Juan José Claros — Dev1

Por qué esta parte es suya: usted es quien más tiempo va a pasar dentro de `shared` y `tenancy` durante todo el proyecto (también posee `billing` desde el Sprint 3), así que el Sprint 1 lo pone a sentar las bases de seguridad que todos los demás van a dar por sentadas.

ID	Tarea	h
`SPIKE-1`	RLS de PostgreSQL + pool Hikari + `@TenantId` + fallo cerrado + prueba de concurrencia	6
`RF-API-02.1`	Módulo `shared`: RFC 9457 + filtro único de correlación	2
`RF-SEG-01.1`	Seguridad base: resource server + meta-test 401	1
`RF-SEG-05.1`	Cifrado por campo: `KeyProvider` + DEK por tenant	2
`HAB-03`	Módulo `tenancy`: registro maestro del tenant	1

Cómo empezar (`SPIKE-1`). El objetivo es que, al final del día, dos tenants con 50 hilos cada uno escribiendo al mismo tiempo **nunca** vean una fila del otro.

1. Cree la migración base con la política RLS del capítulo 5 para una tabla de prueba.
2. Configure el filtro que llama a `set_config('app.tenant_id', ?, true)` al abrir cada transacción (un `TransactionSynchronization` o un aspecto `@Before` sobre el caso de uso).
3. Escriba la prueba de concurrencia: dos `ExecutorService`, 50 hilos cada uno, tenant A y tenant B, todos insertando y leyendo al mismo tiempo. Cuente filas cruzadas al final: debe dar **0**.
4. Verifique el fallo cerrado: abra una conexión sin fijar `app.tenant_id` y confirme que un `SELECT` devuelve 0 filas y un `INSERT` falla.

⚠ Si se agota el timebox de 6 horas

Plan B: usar solo `@TenantId` de Hibernate más un filtro de aplicación, y dejar RLS únicamente en las tablas más sensibles (perfil, decisión, auditoría) para completarla en el Sprint 2. Avísele a Nicole y quede anotado en el registro de deuda.

9.2 Alvaro — Integraciones, consentimiento y datos sintéticos

Alvaro Torrez — Dev2

Por qué esta parte es suya: usted es el dueño de `integration`, `consent` y de `customer` (el Perfil 360) durante todo el proyecto. El Sprint 1 solo construye el primer conector (buró); banco y billetera llegan en el Sprint 2 con el mismo patrón que usted deja armado ahora.

ID	Tarea	h
`SPIKE-4`	Generador sintético coherente (con Nicole): 50 personas consultables en el simulador de buró	4
`HAB-02`	CI: pipeline con gates (build, ArchUnit, Modulith verify, gate de omitidos)	2
`RF-CON-01.1`	Módulo `consent`: alta de consentimiento + evidencia + punto de aplicación en conectores	3
`RF-INT-02.1`	Conector de buró contra WireMock (capa anticorrupción + mapeo al canónico)	2

Cómo empezar (`RF-INT-02.1`). El simulador de buró responde **XML**, a propósito: si su conector solo supiera leer JSON, no estaría probando nada.

```
// stub de WireMock (simulators/wiremock/bureau/mappings/informe.json)
{
  "request": { "method": "GET", "urlPattern": "/bureau/v1/informe/.*" },
  "response": {
    "status": 200,
    "headers": { "Content-Type": "application/xml" },
    "bodyFileName": "informe-buen-pagador.xml"
  }
}
```

Su conector implementa el contrato de la SPI (§2, capítulo 10 del informe): `obtener(...)` llama al simulador por HTTP, `mapear(...)` traduce el XML al fragmento canónico. Nunca cree un `if (modoLocal)` dentro del conector: si necesita datos antes de que el simulador esté listo, pida el *mock* generado desde el contrato del buró.

Cómo empezar (`RF-CON-01.1`). Antes de que `integration` llame a cualquier fuente, debe preguntarle a `consent` si hay una autorización vigente para esa capacidad. Sin eso, la respuesta es `422 CONSENT_REQUIRED` y **no se hace la llamada**.

9.3 Jhon — Dominio de crédito: solicitante, score y auditoría

Jhon Huayhua — Dev3

Por qué esta parte es suya: usted es el dueño de `assessment` (la orquestación de la evaluación), `recommendation` y `audit` durante todo el proyecto, y hace de respaldo en `scoring`. El Sprint 1 le da lo mínimo de cada uno: el scorecard básico y el esqueleto de la orquestación.

ID	Tarea	h
`RF-SEG-03.1`	Auditoría solo inserción: tabla append-only + trigger + prueba de manipulación	1
`RF-PER-01.1`	Alta de solicitante (datos declarados)	1
`RF-SCO-01.1`	Scorecard básico en Java (0–1000, bandas A–E, 3 razones)	2
`HAB-04`	Módulo `assessment`: orquestación síncrona (presupuesto de 2 s)	2

Cómo empezar (`RF-SEG-03.1`). La auditoría tiene que ser *evidente ante manipulación*, no solo "una tabla más".

```
CREATE TABLE audit.access_log (
  id UUID PRIMARY KEY, tenant_id UUID NOT NULL,
  actor TEXT NOT NULL, action TEXT NOT NULL,
  correlation_id TEXT NOT NULL, created_at TIMESTAMPTZ DEFAULT now()
);
REVOKE UPDATE, DELETE, TRUNCATE ON audit.access_log FROM doggy_app;
GRANT INSERT, SELECT ON audit.access_log TO doggy_app;
CREATE TRIGGER audit_no_edit BEFORE UPDATE OR DELETE ON audit.access_log
  EXECUTE FUNCTION forbid_write();
```

Prueba mínima: con un rol elevado, intenten un `UPDATE` sobre esa tabla y confirmen que el *trigger* lo rechaza.

Cómo empezar (`RF-SCO-01.1`). El scorecard es una tabla de puntos, **versionada como datos** (no si-entonces sueltos en Java), para que funcione aunque el modelo ONNX de Nicole todavía no esté listo:

```
# scorecard/v1.yaml
bandas: [{min: 750, letra: A}, {min: 600, letra: B}, {min: 450, letra: C},
         {min: 300, letra: D}, {min: 0,   letra: E}]
reglas:
  - variable: mora_max_12m
    puntos: {sin_mora: 0, "1_30_dias": -80, "31_90_dias": -200}
  - variable: antiguedad_actividad_meses
    puntos: {"menos_6": -150, "6_24": -40, "mas_24": 0}
```

El evaluador en Java lee este YAML, no lo hardcodea: así Jhon o cualquier otra persona puede ajustar la tabla de puntos sin recompilar.

9.4 Nicole — ML, Technical Lead y Scrum Master

Nicole Rodas — Dev4, Technical Lead y Scrum Master

Su rol esta semana tiene dos capas. Como **Dev4** construye el spike de ML igual que el resto del equipo construye el suyo. Como **Technical Lead / Scrum Master** hace, además, el trabajo que el resto del equipo *no* hace: inicializar los repositorios, declarar los secretos y publicar los contratos. Ese trabajo adicional no está dentro de las 55 horas del Sprint 1 Backlog —es el costo de llevar el timón, y está bien que así sea.

ID	Tarea	h
TL-00	Inicializar el monorepo, generar los secretos y el `.env` real, publicar `contracts/` v0	previo
`HAB-01`	Estructura del monorepo + OpenAPI skeleton + catálogo de errores + `seed/tenants.yaml`	3
`HAB-07`	Perfil Financiero 360 v1 (JSON Schema), entregado el día 3	1
`SPIKE-3`	ML de punta a punta: dataset sintético → regresión logística → ONNX → JVM, con Jhon	8

Trabajo previo (*TL-00*), antes de que el equipo llegue el día 1:

```
mkdir -p doggycredit/{contracts/{openapi,errors,events},backend,web,seed,infra,docs}
cd doggycredit && git init && git add . && git commit -m "chore: esqueleto inicial"

cp .env.example .env
# .env queda SOLO en su maquina y en los secrets de CI, nunca en git
echo ".env" >> .gitignore

# secrets a generar y declarar (ejemplo, valores reales solo en su .env):
# DC_DB_PASSWORD=...
# DC_KEYCLOAK_ADMIN_PASSWORD=...
# DC_MASTER_KEY=...              (clave maestra que envuelve la DEK por tenant)
```

Su responsabilidad exclusiva

Nadie más en el equipo genera claves ni las declara. Si alguien necesita una variable nueva, se la pide a usted; usted la agrega a `.env.example` (con un valor de ejemplo, nunca el real) y, si hace falta, a su `.env` local y a los *secrets* del CI.

Contratos v0 (parte de `HAB-01`): antes de la reunión de las 09:00, publique al menos el esqueleto de `contracts/openapi/doggycredit-api.v1.yaml` (los *paths* de `/consentimientos` y `/credit-assessments`, aunque estén vacíos) y `contracts/errors/catalog.yaml` con los primeros códigos (`CONSENT_REQUIRED`, `QUOTA_EXCEEDED`). El equipo los completa con usted durante la reunión, no los inventa por su cuenta.
Cómo empezar (`SPIKE-3`). El objetivo es que un modelo entrenado en Python y ejecutado en Java den **el mismo resultado**, con una tolerancia de una millonésima.

```
# ml/training/train.py (esquema)
import onnxmltools
from sklearn.linear_model import LogisticRegression
model = LogisticRegression().fit(X_train, y_train)
onnx_model = onnxmltools.convert_sklearn(model, initial_types=[...])
onnxmltools.utils.save_model(onnx_model, "scoring-v1.onnx")
```

Del lado de Java, carguen el mismo archivo `.onnx` con ONNX Runtime, corran el mismo vector de entrada del *golden set* de 20 perfiles, y comparen la probabilidad devuelta por Python contra la de Java: la diferencia debe ser $\leq 10^{-6}$.

Si se agota el timebox de 8 horas

Plan B: guardar los coeficientes de la regresión logística como datos (igual que el scorecard de Jhon) y evaluarlos directamente en Java, sin ONNX Runtime. Se pierde la vitrina técnica del modelo exportado, no la funcionalidad del Sprint 1.

9.5 Franco — Identidad, frontend y plataforma de despliegue

Franco Parra — Dev5

Por qué esta parte es suya: usted es el dueño de `web/` (la SPA), `analytics` y de la calidad de punta a punta (E2E, k6) durante todo el proyecto. El Sprint 1 lo pone además a resolver, junto con Nicole, los dos *spikes* de los que depende todo el mundo: Keycloak y la generación de código desde los contratos.

ID	Tarea	h
SPIKE-2	Keycloak multi-tenant: claim `tenant_id`, realm desde la semilla, PKCE (con Nicole)	6
SPIKE-5	`openapi-generator` con Spring Boot 4 y Angular 22 (con Nicole)	3
SPIKE-6	VM de demo: Caddy con HTTPS, despliegue por etiqueta desde CI	3
HAB-05	SPA Angular: pantalla de login con PKCE contra Keycloak	2

Cómo empezar (SPIKE-2). Nicole le entrega el `.env` con las credenciales de administrador de Keycloak (nunca las genere usted mismo). Con eso:

```
docker compose up -d keycloak
# importar el realm generado desde seed/tenants.yaml
docker exec keycloak /opt/keycloak/bin/kc.sh import \
  --file /opt/keycloak/data/import/doggycredit-realm.json
```

Verifiquen dos cosas: que un usuario humano obtiene un token con `tenant_id` vía PKCE, y que un cliente *M2M* (`client_credentials`) también lo obtiene, con el mismo claim. Ambos se validan contra la API antes de dar el spike por cerrado.

Cómo empezar (HAB-05). Con `angular-auth-oidc-client` y el `clientId` que salga del realm importado:

```
// app.config.ts (esquema)
provideAuth({ config: {
  authority: 'https://localhost/auth/realms/doggycredit',
  clientId: 'doggycredit-web', responseType: 'code',
  scope: 'openid profile', useRefreshToken: true,
}})
```

Cómo empezar (SPIKE-6). No hace falta terminarlo temprano en la semana; solo dejarlo probado una vez para que el viernes no sea una sorpresa:

```
# Caddyfile (esquema)
localhost {
  reverse_proxy /api/* doggycredit-api:8080
  root * /srv/web
  file_server
  tls internal
}
```

10. Operación diaria con Docker Compose

10.1 Rutina de inicio

```
docker compose up -d
docker compose ps            // todos "healthy" antes de seguir
```

10.2 Rutina después de cambiar código

```
docker compose up -d --build doggycredit-api
docker compose logs -f doggycredit-api
```

10.3 Rutina de diagnóstico

```
docker compose ps                       // que servicio no esta healthy
docker compose logs --tail=200 <servicio>
curl -s https://localhost/actuator/health | jq
```

10.4 Volúmenes y reinicio limpio

Atención

`docker compose down -v` borra los volúmenes: la base de datos, Keycloak y todo lo que hayan sembrado. Es la forma correcta de empezar el viernes “desde datos limpios”, pero **nunca** se ejecuta a mitad de semana sin avisar: alguien puede perder horas de datos de prueba armados a mano.

10.5 Logs y correlación útiles

Para seguir una petición específica de punta a punta, busquen su `X-Correlation-Id` (lo devuelve cualquier respuesta, incluso las de error) en Grafana (puerto 3000): ahí aparece la traza completa, con los tiempos de cada módulo y de la llamada al simulador.

11. Plan resumido de los Días 2 a 5

Día	Foco
Día 2	Cierre de los *spikes* que sigan abiertos. Los PR de contrato de `HAB-01` deben estar fusionados. Juanjo y Alvaro empiezan a integrar `consent` con `integration` contra el conector de buró real.
Día 3	Nicole entrega el esquema del Perfil Financiero 360 v1 (`HAB-07`). Jhon conecta `assessment` con el scorecard de Alvaro. Franco tiene el login de la SPA funcionando contra Keycloak.
Día 4	Primera integración cruzada completa: login → consentimiento → evaluación → score, sobre Compose completo. Empiezan las pruebas E2E de Franco. Se corrige lo que la integración revele (siempre aparece algo).
Día 5	Congelamiento de la tarde: solo se corrigen fallos del flujo principal, nada nuevo. Ensayo de la demo (guion del capítulo 13). Review y Retro.

Nota

Este plan es un resumen operativo. El detalle completo de los 4 sprints, con las señales de cada hito (H1 a H6), está en el informe de arquitectura, §16.

12. Troubleshooting para principiantes

12.1 Árbol de decisión rápido

Síntoma	Primera sospecha
Un contenedor queda en `restarting`	Revisen `docker compose logs` de ese contenedor; casi siempre es una variable de `.env` faltante o mal escrita
`401` con un token que ustedes creen válido	El token expiró (dura 5 min) o el `realm` importado no coincide con el que generó el token; vuelvan a iniciar sesión
`404` donde esperaban un `200`	Antes de suponer un bug: ¿el recurso pertenece al tenant del token que están usando? Es el comportamiento correcto, no un error
Una fila no aparece pero saben que existe	Revisen si `app.tenant_id` se está fijando en esa transacción; probablemente sea el fallo cerrado de RLS funcionando
El build falla con un error de Modulith	Un módulo está importando algo de `internal/` de otro. Es intencional: mueva la dependencia a través de la `Api` pública
`oasdiff` rompe el CI	Cambiaron el contrato de forma incompatible; si es necesario, versionen el cambio o avisen en el daily antes de forzarlo
Docker sin memoria / muy lento	Revisen cuántos contenedores tienen levantados a la vez; apaguen los que no estén usando en ese momento

12.2 Cuándo pedir ayuda

Después de **30 minutos** bloqueados en el mismo problema, se pide ayuda: en el chat del equipo, con el mensaje de error completo y lo que ya intentaron. No es una señal de debilidad, es la regla de capacidad del capítulo 1.

13. Definition of Done y preparación de la Review

13.1 Checklist por tarea

☐ Fusionada en `main` por PR revisado, CI verde, 0 tests omitidos no autorizados.
☐ Pruebas de integración con Testcontainers reales (no dobles en memoria).
☐ Si tocó un endpoint: está en el meta-test de aislamiento y en el meta-test 401.
☐ Si tocó una tabla con `tenant_id`: tiene RLS y el test la cubre.
☐ Errores en RFC 9457 con código del catálogo único.
☐ Logs con id de correlación, sin datos personales.
☐ Demostrado en Compose completo, no en la máquina de quien lo escribió.

13.2 Checklist del incremento (cierre del Sprint 1)

☐ El E2E del flujo principal (login → consentimiento → evaluación → score) pasa en CI sobre `main`.
☐ Etiqueta `v0.1.0-s1` desplegada en la VM de demo, desde CI.
☐ Los 6 *spikes* cerraron con “sigue” o con su plan B documentado.
☐ Registro de deuda actualizado con lo que quedó pendiente.

13.3 Guion de la Review del Sprint 1

1. Login de un usuario sembrado del tenant *Progreso Andino*, con PKCE contra Keycloak.
2. Alta de un titular con su consentimiento para el buró.
3. Evaluación que llama por red al simulador del buró y devuelve un score básico con 3 razones.
4. En vivo: una llamada sin token responde `401`; un usuario de otro tenant que pide el mismo titular responde `404`.
5. El mismo `X-Correlation-Id` visible en la respuesta, en Grafana y en la tabla de auditoría.
6. Se apaga el simulador del buró y se repite la evaluación: sale `PARCIAL`, nunca inventada.

A. Hoja rápida de comandos

```
# infraestructura
docker compose up -d
docker compose ps
docker compose logs -f <servicio>
docker compose down -v          # borra datos locales, con acuerdo del equipo

# backend
mvn -pl backend spring-boot:run
mvn -pl backend test
mvn -pl backend verify          # incluye Testcontainers

# contratos
mvn -pl backend generate-sources    # regenera desde contracts/openapi
npm --prefix web run generate:api   # cliente TypeScript

# frontend
npm --prefix web install
npm --prefix web start

# git
git checkout -b feat/DC-101-consent-api
git push -u origin feat/DC-101-consent-api
```

B. Variables y nombres recomendados

Variable (`.env.example`)	Para qué
`DC_DB_HOST` / `DC_DB_PASSWORD`	Conexión a PostgreSQL. La contraseña real solo la declara Nicole.
`DC_KEYCLOAK_ISSUER_URI`	URL del *realm* para validar tokens.
`DC_KEYCLOAK_ADMIN_PASSWORD`	Solo para importar el *realm* localmente; solo Nicole la conoce en valor real.
`DC_MASTER_KEY`	Clave maestra que envuelve la DEK por tenant (cifrado por campo). Exclusiva de Nicole.
`DC_BUREAU_BASE_URL`	URL del simulador WireMock del buró.
`DC_CORRELATION_HEADER`	Nombre de la cabecera de correlación (`X-Correlation-Id`), fijo, no se cambia por módulo.

Atención

Este anexo lista **el nombre** de las variables y para qué sirven. Los valores reales viven únicamente en el `.env` de Nicole y en los *secrets* de CI, nunca en este documento ni en el repositorio.

C. Glosario mínimo

Término	Significado
Tenant	Institución cliente de DoggyCredit, con sus datos aislados de los demás tenants.
Módulo (Spring Modulith)	Paquete Java con una `Api` pública y un `internal/` invisible desde afuera; sus límites se verifican en el build.

RLS	*Row-Level Security* de PostgreSQL: la base filtra filas por tenant aunque la aplicación se equivoque.
Contrato	Documento (OpenAPI, catálogo de errores, esquema de evento) que dos partes del sistema respetan sin negociar en el momento.
Contract-first	Práctica de escribir el contrato antes que el código, y generar código desde él.
Spike	Investigación con tiempo límite (*timebox*) y un criterio de salida claro, con un plan B si no se cumple.
Scorecard	Tabla de puntos por atributo, versionada como datos; explicable por construcción.
ONNX	Formato portable para exportar un modelo entrenado en Python y ejecutarlo dentro de la JVM.
WireMock	Herramienta que simula un servicio HTTP externo (en este sprint, el buró de crédito).
`X-Correlation-Id`	Identificador único de una petición, visible en la respuesta, en los logs y en la auditoría.
DoR / DoD	*Definition of Ready* / *Definition of Done*: criterios para empezar y para dar por terminada una historia.

Referencias y fuentes

- *DoggyCredit — Visión, Requerimientos y Arquitectura v1* (documento de la mesa de diseño: Product Owner, Arquitecto, Senior Software Engineer). Fuente normativa de la arquitectura, las reglas de negocio y los ADR.
- `DoggyCredit_Backlogs.xlsx` — hojas *Product Backlog* y *Sprint 1 - Backlog*. Fuente de verdad de alcance, horas, estado y dependencias.
- *MedicalSys — Guía técnica e inducción del Sprint 2*, primera guía de esta misma serie.

Los números de página se actualizan automáticamente durante la generación del documento.